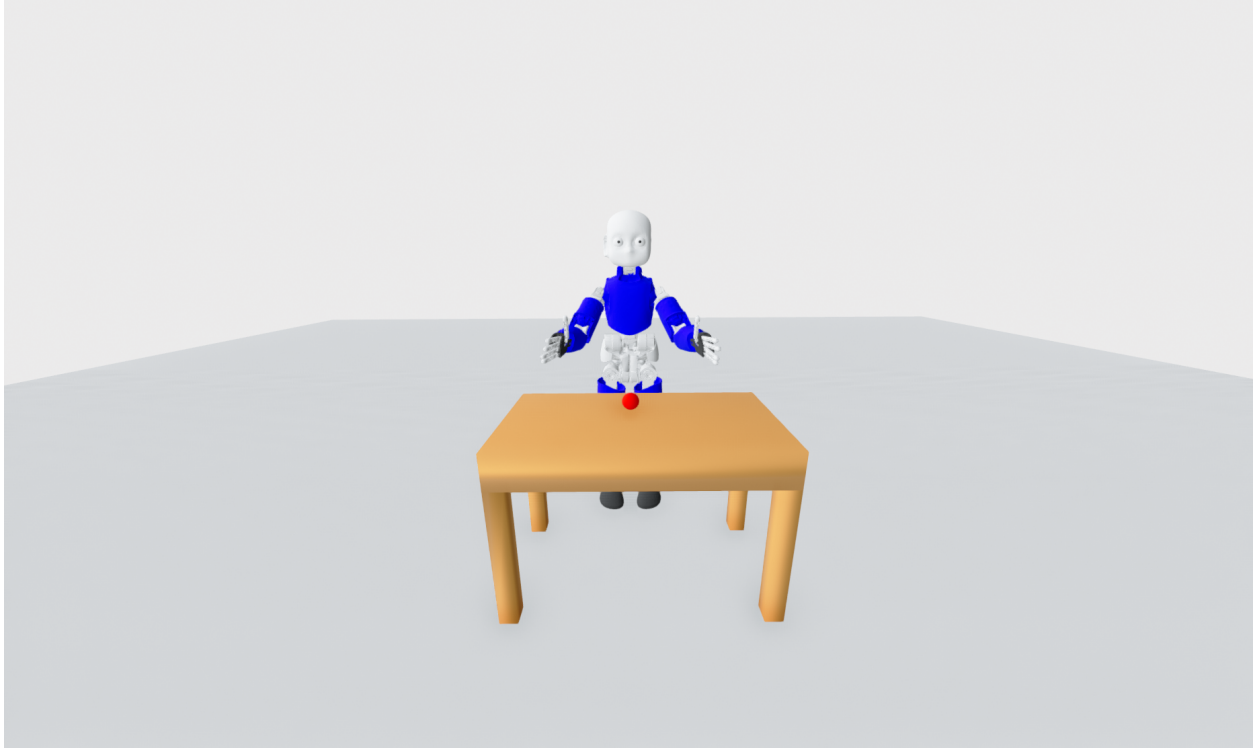


# pyCub

Author: Lukas Rustler



**CONTENTS:**

|          |                                     |           |
|----------|-------------------------------------|-----------|
| <b>1</b> | <b>README</b>                       | <b>1</b>  |
| 1.1      | Installation . . . . .              | 1         |
| 1.2      | Examples . . . . .                  | 2         |
| 1.3      | Information . . . . .               | 2         |
| 1.4      | FAQ . . . . .                       | 2         |
| 1.5      | Docker . . . . .                    | 2         |
| 1.6      | Known bugs . . . . .                | 5         |
| <b>2</b> | <b>icub_pybullet</b>                | <b>7</b>  |
| 2.1      | pyCub . . . . .                     | 7         |
| 2.2      | utils . . . . .                     | 13        |
| 2.3      | visualizer . . . . .                | 15        |
| <b>3</b> | <b>examples</b>                     | <b>19</b> |
| 3.1      | push_the_ball_cartesian . . . . .   | 19        |
| 3.2      | push_the_ball_pure_joints . . . . . | 19        |
| 3.3      | skin_test . . . . .                 | 20        |
|          | <b>Python Module Index</b>          | <b>21</b> |
|          | <b>Index</b>                        | <b>23</b> |



## README

pyCub is iCub humanoid robot simulator written in Python. It uses PyBullet for simulation and Open3D for visualization.

## 1.1 Installation

- Requires python3.8 to python3.11
  - newer python versions are now not supported due to incompatible with some dependencies
- We recommend using virtual environment when installing from PyPi or from source

```
– python3 -m venv pycub_venv
  source pycub_venv/bin/activate
  OTHER_COMMANDS
```

### 1. (Recommended) Install from PyPi (not yet available)

- `python3 -m pip install icub_pybullet`

### 2. Install from source

- Pull this repository

```
• cd PATH_TO_THE_REPOSITORY
  python3 -m pip install --upgrade pip
  python3 -m pip install -r requirements.txt
  python3 setup.py install
```

### 3. Native Docker (GNU/Linux only)

- see *Docker Native Version* section

### 4. VNC Docker

- see *Docker VNC Version* section

### 5. Gitpod

- open <https://gitpod.io/#github.com/rustlluk/pycub> and log in with GitHub account

## 1.2 Examples

- [push\\_the\\_ball\\_pure\\_joints.py](#) contains an example that shows how to control the robot in joint space
- [push\\_the\\_ball\\_cartesian.py](#) contains an example that shows how to control the robot in Cartesian space
- [skin\\_test.py](#) contains an example with balls falling the robot and skin should turn green on the places where contact occurs. You may want to slow the simulation a little bit to see that :)

## 1.3 Information

- documentation can be found at [lukasrustler.cz/pycub\\_documentation](http://lukasrustler.cz/pycub_documentation) or in [pyCub.pdf](#)
- presentation with description of functionality can be found at [pyCub presentation](#)
- simulator code is in [pycub.py](#)
  - it uses PyBullet for simulation and provides high-level interface
- visualization code in [visualizer.py](#)
  - it uses Open3D for visualization as it is much more customizable than PyBullet default GUI

## 1.4 FAQ

1. You get some kind of error with the visualization, e.g., segmentation fault.
  1. Try to check graphics mesa/opengl/nvidia drivers as those are the most common source of problems for the OpenGL visualization
  2. Try Native Docker
  3. In given config your load set in GUI standard=False; web=True to enable web visualization
  4. Try VNC Docker
  5. Try Gitpod Docker
2. You get import errors, e.g. cannot load pycub from icub\_pybullet
  1. Install from pip with `python3 -m pip install icub_pybullet`
  2. Install the package correctly with `python3 setup.py install` from root directory of this repository
  3. put `icub_pybullet` directory to your PYTHONPATH

## 1.5 Docker

### 1.5.1 Native Version

- version with native GUI; useful when you have problems with OpenGL (e.g., usually some driver issues)
- only for GNU/Linux systems (Ubuntu, Mint, Arch, etc.)
  1. install [docker-engine](#) (**DO NOT INSTALL DOCKER DESKTOP**)
    - **perform** [post-installation steps](#)

### 1. Build/Pull the Docker Image

- clone this repository

```
cd SOME_PATH
git clone https://github.com/rustlluk/pyCub.git
```

- pull the docker image (see *Parameters* for more parameters)

```
cd SPATH_TO_THE_REPOSITORY/Docker
./deploy.py -p PATH_TO_THE_REPOSITORY -c pycub -pu
```

- or, build the docker (see *Parameters* for more parameters)

```
cd SOME_PATH/pycub_ws/Docker
./deploy.py -p PATH_TO_THE_REPOSITORY -c pycub -b
```

- after you pull or build the container, you can run it next time as

```
./deploy.py -c pycub -e
```

- if you want to open new terminal in existing container, run

```
./deploy.py -c pycub -t
```

#### ### VNC Version

- this version works also on Windows and MacOS because it uses VNC server to show the GUI, i.e., the output will be shown on <http://localhost:6080>

### 1. Install [docker-engine](#) (GNU/Linux only) or [docker-desktop](#) (all systems)

- **perform** [post-installation steps](#)

### 1. The same as for *VNC Version*, but use `-vnc` option, e.g., to pull and run the image

```
cd PATH_TO_THE_REPOSITORY/Docker
./deploy.py -p PATH_TO_THE_REPOSITORY -c pycub -pu -vnc
```

### 2. run `start-vnc-sessions.sh` script in the container

### 3. Open <http://localhost:6080>

## 1.5.2 Docker + PyCharm

### Native Version:

You have two option:

1. Either run pycharm from docker
2. Open your pycharm on your host machine:
  - add ssh interpreter
    - user docker
    - ip can be localhost or ip where you run the docker
    - port 2222

- uncheck automatic upload to remote folder
  - change remote path to /home/docker/pycub\_ws ##### VNC/Gitpod version
3. open pycharm from inside the container

### 1.5.3 Deploy Parameters

- cd to folder with Dockerfile
- ./deploy.py
  - -b or --build when building
    - \* default: False
  - -e if you just want to run existing docker without building
    - \* default: False
  - -p or --path with path to current folder
    - \* default: ""
  - -pu or --pull to pull the image from dockerhub
    - \* default: False
  - -c or --container with desired name of the new, created container
    - \* default: my\_new\_docker
  - -t or --terminal to run new terminal in running docker session
    - \* default: False
  - -pv or --python-version to specify addition python version to install
    - \* default: 3.11
  - -pcv or --pycharm-version to specify version of pycharm to use
    - \* default: 2023.2.3
  - -bi or --base-image to specify base image that will be used
    - \* default: ubuntu:20.04
    - \* other can be found at [hub.docker.com](https://hub.docker.com)

Do this on computer where you will run the code. If you have a server you have to run it on the server over SSH to make things work properly.

### 1.5.4 Docker FAQ

- **you get error of not being in sudo group when running image**
  - check output of `id -u` command. If the output is not 1000 you have to build the image by yourself and can not pull it
    - \* this happens when your account is not the first one created on your computer
- **``sudo apt install something`` does not work**
  - you need to run `sudo apt update` first after you run the container for the first time

\* apt things are removed in Dockerfile, so it does not take unnecessary space in the image

## 1.6 Known bugs

- visualization with skin dies after ~65k steps
  - e.g., <https://github.com/isl-org/Open3D/issues/4992>





## ICUB\_PYBULLET

### 2.1 pyCub

**class** icub\_pybullet.pycub.**EndEffector**(*name: str, client: int*)

Bases: object

Help function for end-effector encapsulation

#### Parameters

- **name** (*str*) – name of the end-effector
- **client** (*pointer to pyCub instance*) – parent client

**get\_position**() → *Pose*

Function to get current position of the end-effector

**class** icub\_pybullet.pycub.**Joint**(*name: str, robot\_joint\_id: int, joints\_id: int, lower\_limit: float, upper\_limit: float, max\_force: float, max\_velocity: float*)

Bases: object

Help class to encapsulate joint information

#### Parameters

- **name** (*str*) – name of the joint
- **robot\_joint\_id** (*int*) – id of the joint in pybullet
- **joints\_id** (*int*) – id of the joint in pycub.joints
- **lower\_limit** (*float*) – lower limit of the joint
- **upper\_limit** (*float*) – upper limit of the joint
- **max\_force** (*float*) – max force of the joint
- **max\_velocity** (*float*) – max velocity of the joint

**class** icub\_pybullet.pycub.**Link**(*name: str, robot\_link\_id: int, urdf\_link: int*)

Bases: object

Help function to encapsulate link information

#### Parameters

- **name** (*str*) – name of the link
- **robot\_link\_id** (*int*) – id of the link in pybullet

- **urdf\_link** (*int*) – id of the link in `pycub.urdfs["robot"].links`

**class** `icub_pybullet.pycub.pyCub`(*config: str | None = 'default.yaml'*)

Bases: `BulletClient`

Client class which inherits from `BulletClient` and contains the whole simulation functionality

**Parameters**

**config** (*str*, *optional*, *default="default.yaml"*) – path to the config file

**static bbox\_overlap**(*b1\_min: list*, *b1\_max: list*, *b2\_min: list*, *b2\_max: list*) → *bool*

Function to check whether two bounding boxes overlap

**Parameters**

- **b1\_min** (*list*) – min bbox 1
- **b1\_max** (*list*) – max bbox 1
- **b2\_min** (*list*) – min bbox 2
- **b2\_max** (*list*) – max bbox 2

**Returns**

whether the boxes overlap

**Return type**

*bool*

**compute\_jacobian**(*chain: Literal['left\_arm', 'right\_arm', 'left\_leg', 'right\_leg', 'torso', 'head']*, *start: str | None = None*, *end: str | None = None*) → *Tuple[array, array]*

Help function to compute a Jacobian using RTB of a given chain with optional start and end

**Parameters**

- **chain** (*str*) – name of the chain
- **start** (*str*) – name of the start link
- **end** (*str*) – name of the end link

**Returns**

Jacobian and list of joints used

**Return type**

*np.array, np.array*

**compute\_skin**() → *None*

Function to emulate skin activations using ray casting.

**contactPoints** = {'DISTANCE': 8, 'FLAG': 0, 'FORCE': 9, 'FRICTION1': 10, 'FRICTION2': 12, 'FRICTIONDIR1': 11, 'FRICTIONDIR2': 13, 'IDA': 1, 'IDB': 2, 'INDEXA': 3, 'INDEXB': 4, 'NORMAL': 7, 'POSITIONA': 5, 'POSITIONB': 6}

**create\_urdf**(*object\_path: str*, *fixed: bool*, *color: List[float]*, *suffix: str | None = ""*)

Creates a URDF for the given .obj file

**Parameters**

- **object\_path** (*str*) – path to the .obj
- **fixed** (*bool*) – whether the object is fixed in space
- **color** (*list of 3 floats*) – color of the object

- **suffix**(*str*, *optional*, *default=""*) – suffix to add to the object name

```
dynamicsInfo = {'BODYTYPE': 10, 'DAMPING': 8, 'FRICTION': 1, 'INERTIAOR': 4,
'INERTIAPOS': 3, 'INTERTIADIAGONAL': 2, 'MARGIN': 11, 'MASS': 0, 'RESTITUTION': 5,
'ROLLINGFRICTION': 6, 'SPINNINGFRICTION': 7, 'STIFFNESS': 9}
```

**find\_joint\_id**(*joint\_name: JOINTS | JOINTS\_IDS*) → Tuple[int, int]

Help function to get indexes from joint name of joint index in self.joints list

**Parameters**

**joint\_name** (*str* or *int*) – name or index of the link

**Returns**

joint id in pybullet and pycub space

**Return type**

int, int

**find\_link\_id**(*mesh\_name: str*, *robot: int | None = None*, *urdf\_name: str | None = 'robot'*) → int

Help function to find link id from mesh name

**Parameters**

- **mesh\_name** (*str*) – name of the mesh (only basename with extension)
- **robot** (*int*, *optional*, *default=None*) – robot pybullet id
- **urdf\_name** (*str*, *optional*, *default="robot"*) – name of the object in pycub.urdfs

**Returns**

id of the link in pybullet space

**Return type**

int

**find\_processes\_by\_name**() → List[int]

Help function to find PIDs of processes with the parent name

**Returns**

list of matching PIDs

**Return type**

list of ints

**get\_camera\_images**() → List[array]

Gets the images from enabled eye cameras

**Returns**

list of images from enabled cameras

**Return type**

list of numpy array

**static get\_chains**() → Tuple[dict, dict]

Function to get chains (and corresponding chains of joints) of the robot

**Returns**

chains of links and chains of joints

**Return type**

dict, dict

**get\_joint\_state**(*joints: JOINTS | List[JOINTS] | JOINTS\_IDS | List[JOINTS\_IDS] | None = None,*  
*allow\_error: bool | None = False*) → List[list]

Get the state of the specified joints

**Parameters**

- **joints** (*int or list, optional, default=None*) – joint or list of joints to get the state of
- **allow\_error** (*bool, optional, default=False*) – whether to allow errors (non-existing joints); useful for Jacobian computation

**Returns**

list of states of the joints

**Return type**

list

**init\_robot**() → Tuple[int, List[*Joint*], List[*Link*]]

Load the robot URDF and get its joints' information

**Returns**

robot, its joints and links

**Return type**

int, list of *Joint*, list of *Link*

**init\_urdfs**() → None

Function to load URDFs of other objects

**Returns**

**Return type**

**is\_alive**() → bool

Checks whether the engine is still running

**Returns**

True when running

**Return type**

bool

```
jointInfo = {'AXIS': 13, 'DAMPING': 6, 'FLAGS': 5, 'FRICTION': 7, 'INDEX': 0,  
'LINKNAME': 12, 'LOWERLIMIT': 8, 'MAXFORCE': 10, 'MAXVELOCITY': 11, 'NAME': 1,  
'PARENTINDEX': 16, 'PARENTORN': 15, 'PARENTPOS': 14, 'QINDEX': 3, 'TYPE': 2,  
'UINDEX': 4, 'UPPERLIMIT': 9}
```

```
jointStates = {'FORCES': 2, 'POSITION': 0, 'TORQUE': 3, 'VELOCITY': 1}
```

**kill\_open3d**() → None

A bit of a workaround to kill open3d, that seems to hang for some reason.

**Returns**

**Return type**

```
linkInfo = {'ANGVEL': 7, 'INERTIAORI': 3, 'INERTIAPOS': 2, 'LINVEL': 6, 'URDFORI':  
5, 'URDFPOS': 4, 'WORLDORI': 1, 'WORLDPOS': 0}
```

**motion\_done**(*joints: JOINTS | List[JOINTS] | JOINTS\_IDS | List[JOINTS\_IDS] | None = None, check\_collision=True*) → bool

Checks whether the motion is done.

#### Parameters

- **joints** (*int or list, optional, default=None*) – joint or list of joints to get the state of
- **check\_collision** (*bool, optional, default=True*) – whether to check for collision during motion

#### Returns

True when motion is done, false otherwise

#### Return type

bool

**move\_cartesian**(*pose: Pose, wait: bool | None = True, velocity: float | None = 1, check\_collision: bool | None = True*) → None

Move the robot in cartesian space by computing inverse kinematics and running position control

#### Parameters

- **pose** (*utils.Pose*) – desired pose of the end effector
- **wait** (*bool, optional, default=True*) – whether to wait for movement completion
- **velocity** (*float, optional, default=1*) – joint velocity to move with
- **check\_collision** (*bool, optional, default=True*) – whether to check for collisions during motion

**move\_position**(*joints: JOINTS | List[JOINTS] | JOINTS\_IDS | List[JOINTS\_IDS], positions: float | List[float], wait: bool | None = True, velocity: float | None = 1, set\_col\_state: bool | None = True, check\_collision: bool | None = True*) → None

Move the specified joints to the given positions

#### Parameters

- **joints** (*int, str, list of int, list of str*) – joint or list of joints to move
- **positions** (*float or list; same length as joints*) – position or list of positions to move the joints to
- **wait** (*bool, optional, default=True*) – whether to wait until the motion is done
- **velocity** (*float, optional, default=1*) – velocity to move the joints with
- **set\_col\_state** (*bool, optional, default=True*) – whether to reset collision state
- **check\_collision** (*bool, optional, default=True*) – whether to check for collision during motion

**move\_velocity**(*joints: JOINTS | List[JOINTS] | JOINTS\_IDS | List[JOINTS\_IDS], velocities: float | List[float]*) → None

Move the specified joints with the specified velocity

#### Parameters

- **joints** (*int or list*) – joint or list of joints to move
- **velocities** (*float or list; same length as joints*) – velocity or list of velocities to move the joints to

**prepare\_log()** → str

Prepares the log string

**Returns**

log string

**Return type**

str

**print\_collision\_info**(*c: list | None = None*)

Help function to print collision info

**Parameters**

**c** (*list*, *optional*, *default=None*) – one collision; if None print all collisions

**run\_vhacd**(*robot: bool | None = True*) → None

Function to run VHACD on all objects in loaded URDFs, and to create new URDFs with changed collision meshes

**Parameters**

**robot** (*bool*, *optional*, *default=True*) – whether to run VHACD on the robot

**static scale\_bbox**(*bbox: list*, *scale: float*) → Tuple[array, array]

Function to scale the bounding box

**Parameters**

- **bbox** (*list*) – list of min and max bbox
- **scale** (*float*) – scale factor

**Returns**

new min and max bbox

**Return type**

list, list

**stop\_robot()** → None

Stops the robot

**toggle\_gravity()** → None

Toggles the gravity

**update\_simulation**(*sleep\_duration: float | None | None = -1*) → None

Updates the simulation

**Parameters**

**sleep\_duration** (*float or None*, *optional*, *default=-1*) – duration to sleep before the next simulation step

**visualShapeData** = {'COLOR': 7, 'DIMS': 3, 'FILE': 4, 'GEOMTYPE': 2, 'ID': 0, 'LINK': 1, 'ORI': 6, 'POS': 5, 'TEXTURE': 8}

**wait\_motion\_done**(*sleep\_duration: float | None = 0.01*, *check\_collision: bool | None = True*) → None

Help function to wait for motion to be done. Can sleep for a specific duration

**Parameters**

- **sleep\_duration** (*float*, *optional*, *default=0.01*) – how long to sleep before running simulation step
- **check\_collision** (*bool*, *optional*, *default=True*) – whether to check for collisions during motion

## 2.2 utils

**class** icub\_pybullet.utils.**Config**(*config\_path: str*)

Bases: object

Class to parse and keep the config loaded from yaml file

### Parameters

**config\_path** (*str*) – path to the config file

**set\_attribute**(*attr: str, value: Any, reference: int*) → int

Function to recursively fill the instance variables from dictionary. When value is non-dict, it is directly assigned to a variable. Else, the dict is recursively parsed.

### Parameters

- **attr** (*str*) – name of the attribute
- **value** (*str, float, int, dict, list, ... - and other that can be loaded from yaml*) – value of the attribute
- **reference** (*pointer or whatever it is called in Python*) – reference to the parent class. “self” for the upper attributes, pointer to namedtuple for inner attributes

### Returns

0

### Return type

int

**class** icub\_pybullet.utils.**CustomFormatter**(*fmt=None, datefmt=None, style='%', validate=True*)

Bases: Formatter

Custom formatter that assigns colors to logs From <https://stackoverflow.com/a/56944256>

Initialize the formatter with specified format strings.

Initialize the formatter either with the specified format string, or a default as described above. Allow for specialized date formatting with the optional datefmt argument. If datefmt is omitted, you get an ISO8601-like (or RFC 3339-like) format.

Use a style parameter of ‘%’, ‘{’ or ‘\$’ to specify that you want to use one of %-formatting, `str.format()` ({}), formatting or `string.Template` formatting in your format string.

Changed in version 3.2: Added the style parameter.

```
FORMATS = {10: '\x1b[38;20m%(module)s %(levelname)s: %(message)s\x1b[0m', 20:
'\x1b[38;20m%(module)s %(levelname)s: %(message)s\x1b[0m', 30:
'\x1b[33;20m%(module)s %(levelname)s: %(message)s\x1b[0m', 40:
'\x1b[31;20m%(module)s %(levelname)s: %(message)s\x1b[0m', 50:
'\x1b[31;1m%(module)s %(levelname)s: %(message)s\x1b[0m'}
```

```
bold_red = '\x1b[31;1m'
```

**format**(*record: LogRecord*) → str

Format the specified record as text.

The record’s attribute dictionary is used as the operand to a string formatting operation which yields the returned string. Before formatting the dictionary, a couple of preparatory steps are carried out. The message attribute of the record is computed using `LogRecord.getMessage()`. If the formatting string uses the time



(as determined by a call to `usesTime()`, `formatTime()` is called to format the event time. If there is exception information, it is formatted using `formatException()` and appended to the message.

```
grey = '\x1b[38;20m'
```

```
red = '\x1b[31;20m'
```

```
reset = '\x1b[0m'
```

```
yellow = '\x1b[33;20m'
```

```
class icub_pybullet.utils.Pose(pos: list, ori: list)
```

Bases: object

Mini help class for Pose representation

Init function that takes position and orientation and saves them as attributes

**Parameters**

- **pos** (*list*) – x,y,z position
- **ori** (*list*) – rpy orientation

**to\_string()** → str

```
class icub_pybullet.utils.URDF(path: str)
```

Bases: object

Class to parse URDF file

**Parameters**

**path** (*str*) – path to the URDF file

**ROOT\_TAGS** = []

**dereference()** → None

Make parent/child again as names to allow urdf write

**find\_root\_tags()** → None

Finds tags that are 'root', i.e., they have child 'inside'

**fix\_urdf()** → None

Fix the URDF file by converting non-mesh geometries to mesh and saving them as .obj files. If changes were made, write the new URDF to a file.

**make\_references()** → None

Make parent/child in joint list as references to the given link

**read**(*el: Element, parent: Element*) → None

Recursive function to read the URDF file. When there are no children, it reads the attributes and saves them.

**Parameters**

- **el** (*xml.etree.ElementTree.Element*) – The current element in the XML tree.
- **parent** (*xml.etree.ElementTree.Element*) – The parent element in the XML tree.

**write\_attr**(*attr\_name: str, attr: Any, level: int | None = 1, skip\_header: bool | None = False*) → None

Write an attribute to the new URDF string.

#### Parameters

- **attr\_name** (*str*) – The name of the attribute.
- **attr** (*any*) – The attribute value.
- **level** (*int, optional, default=1*) – The indentation level for the attribute.
- **skip\_header** (*bool, optional, default=False*) – Whether to skip writing the attribute header.

**write\_urdf**() → None

Write the URDF object to a string.

## 2.3 visualizer

**class** `icub_pybullet.visualizer.Visualizer`(*client: int*)

Bases: object

Class to help with custom rendering

#### Parameters

**client** (*int*) – Pointer to the PyCub class instance

**class** `EyeWindow`(*eye: str, parent: int*)

Bases: object

Class to handle windows for eye rendering

#### Parameters

- **eye** (*str*) – name of the eye
- **parent** (*int*) – The parent class (Visualizer).

**MENU\_IDS** = {'l\_eye': [2, 3, 8], 'r\_eye': [4, 5, 9]}

**POSITIONS** = {'l\_eye': [320, 560], 'r\_eye': [0, 560]}

**get\_image**() → None

Small function to get image from open3d

#### Returns

**Return type**

**on\_close**() → bool

Small function to delete the window from the parent class

**on\_mouse**(*event: MouseEvent*) → int

Small function to ignore mouse events

#### Parameters

**event** (*gui.MouseEvent*) – Mouse event

**save\_image**(*im: Image*) → None

Callback to get images from open3d

#### Parameters

**im** (*o3d.geometry.Image*) – the image to be saves

**save\_images()** → None

Function to save stream of images to file

**class MenuCallback**(*menu\_id: int, parent: int*)

Bases: object

Class to handle menu callbacks.

Initialize the MenuCallback class.

**Parameters**

- **menu\_id** (*int*) – The id of the menu.
- **parent** (*pointer to the class of visualizer.Visualizer type*) – The parent class (Visualizer).

**input\_completed**(*text: str | None = None*)

Callback for the dialog

**Parameters**

**text** (*str*) – input text

**Returns**

**Return type**

**save\_image**(*im: Image, mode: int*) → None

Save the image. It shows FileDialog to find path for image save. It saves it with the current resolution of the window.

**Parameters**

- **im** (*o3d.geometry.Image*) – The image to be saved.
- **mode** (*int*) – The mode of the image. 0 for RGB, 1 for depth.

**wait\_for\_dialog\_completion()** → None

Help function to keep the gui loop running

**find\_xyz\_rpy**(*mesh\_name: str, urdf\_name: str | None = 'robot'*) → Tuple[list, list, float, str]

Find the xyz, rpy and scales values.

**Parameters**

- **mesh\_name** (*str*) – The name of the mesh.
- **urdf\_name** (*str, optional, default="robot"*) – The name of the urdf.

**Returns**

The xyz, rpy, and scales, link\_name

**Return type**

list, list, float, str

**read\_info**(*obj\_id: int*) → int

Read info from PyBullet

**Parameters**

**obj\_id** (*int*) – id of the object; given by pybullet

**Returns**

0 for success

**Return type**

int

**render()** → None

Render all the things

**show\_first**(*urdf\_name: str | None = 'robot'*) → None

Show the first batch of meshes in the visualizer. It loads the meshes and saves the to dict for quicker use later

**Parameters**

**urdf\_name** (*str, optional, default="robot"*) – The name of the urdf to be used.

**show\_mesh()** → None

Function to parse info about meshes from PyBullet



## EXAMPLES

### 3.1 push\_the\_ball\_cartesian

Example of moving the robot in cartesian space to push the ball. It is more robust than the pure joint control.

**Author**

Lukas Rustler

`icub_pybullet.examples.push_the_ball_cartesian.main()` → NoReturn

Main function to run the example

**Returns**

**Return type**

`icub_pybullet.examples.push_the_ball_cartesian.push_the_ball(client: pyCub)` → None

Example function to move the ball with cartesian control.

**Parameters**

**client** (`icub_pybullet.pycub.pyCub`) – instance of `pyCub`

**Returns**

**Return type**

### 3.2 push\_the\_ball\_pure\_joints

Example of how to push the ball from the table using only pure joint control. It works without planner of collisions detection/avoidance. It is not very robust, and it is laborious, but it is a good starting point for your own experiments.

**Author**

Lukas Rustler

`icub_pybullet.examples.push_the_ball_pure_joints.main()` → NoReturn

Main function to run the example

**Returns**

**Return type**

`icub_pybullet.examples.push_the_ball_pure_joints.push_the_ball(client: pyCub)` → None

Example function to push the ball from the table with joint control.

**Parameters**

**client** (`icub_pybullet.pycub.pyCub`) – instance of `pyCub`

**Returns**

**Return type**

### 3.3 skin\_test

Script to test the skin sensors. Balls falling to the skin and turning activated point to green should be seen.

**Author**

Lukas Rustler

`icub_pybullet.examples.skin_test.main()` → NoReturn

Main function to run the example

**Returns**

**Return type**

## PYTHON MODULE INDEX

### i

`icub_pybullet.examples.push_the_ball_cartesian`,  
19  
`icub_pybullet.examples.push_the_ball_pure_joints`,  
19  
`icub_pybullet.examples.skin_test`, 20  
`icub_pybullet.pycub`, 7  
`icub_pybullet.utils`, 13  
`icub_pybullet.visualizer`, 15





## INDEX

### B

`bbox_overlap()` (*icub\_pybullet.pycub.pyCub* static method), 8  
`bold_red` (*icub\_pybullet.utils.CustomFormatter* attribute), 13

### C

`compute_jacobian()` (*icub\_pybullet.pycub.pyCub* method), 8  
`compute_skin()` (*icub\_pybullet.pycub.pyCub* method), 8  
`Config` (class in *icub\_pybullet.utils*), 13  
`contactPoints` (*icub\_pybullet.pycub.pyCub* attribute), 8  
`create_urdf()` (*icub\_pybullet.pycub.pyCub* method), 8  
`CustomFormatter` (class in *icub\_pybullet.utils*), 13

### D

`dereference()` (*icub\_pybullet.utils.URDF* method), 14  
`dynamicsInfo` (*icub\_pybullet.pycub.pyCub* attribute), 9

### E

`EndEffector` (class in *icub\_pybullet.pycub*), 7

### F

`find_joint_id()` (*icub\_pybullet.pycub.pyCub* method), 9  
`find_link_id()` (*icub\_pybullet.pycub.pyCub* method), 9  
`find_processes_by_name()` (*icub\_pybullet.pycub.pyCub* method), 9  
`find_root_tags()` (*icub\_pybullet.utils.URDF* method), 14  
`find_xyz_rpy()` (*icub\_pybullet.visualizer.Visualizer* method), 16  
`fix_urdf()` (*icub\_pybullet.utils.URDF* method), 14  
`format()` (*icub\_pybullet.utils.CustomFormatter* method), 13  
`FORMATS` (*icub\_pybullet.utils.CustomFormatter* attribute), 13

### G

`get_camera_images()` (*icub\_pybullet.pycub.pyCub* method), 9  
`get_chains()` (*icub\_pybullet.pycub.pyCub* static method), 9  
`get_image()` (*icub\_pybullet.visualizer.Visualizer.EyeWindow* method), 15  
`get_joint_state()` (*icub\_pybullet.pycub.pyCub* method), 9  
`get_position()` (*icub\_pybullet.pycub.EndEffector* method), 7  
`grey` (*icub\_pybullet.utils.CustomFormatter* attribute), 14

### I

`icub_pybullet.examples.push_the_ball_cartesian` module, 19  
`icub_pybullet.examples.push_the_ball_pure_joints` module, 19  
`icub_pybullet.examples.skin_test` module, 20  
`icub_pybullet.pycub` module, 7  
`icub_pybullet.utils` module, 13  
`icub_pybullet.visualizer` module, 15  
`init_robot()` (*icub\_pybullet.pycub.pyCub* method), 10  
`init_urdfs()` (*icub\_pybullet.pycub.pyCub* method), 10  
`input_completed()` (*icub\_pybullet.visualizer.Visualizer.MenuCallback* method), 16  
`is_alive()` (*icub\_pybullet.pycub.pyCub* method), 10

### J

`Joint` (class in *icub\_pybullet.pycub*), 7  
`jointInfo` (*icub\_pybullet.pycub.pyCub* attribute), 10  
`jointStates` (*icub\_pybullet.pycub.pyCub* attribute), 10

### K

`kill_open3d()` (*icub\_pybullet.pycub.pyCub* method), 10

## L

`Link` (class in `icub_pybullet.pycub`), 7  
`linkInfo` (`icub_pybullet.pycub.pyCub` attribute), 10

## M

`main()` (in module `icub_pybullet.examples.push_the_ball_cartesian`), 19  
`main()` (in module `icub_pybullet.examples.push_the_ball_pure_joints`), 19  
`main()` (in module `icub_pybullet.examples.skin_test`), 20  
`make_references()` (`icub_pybullet.utils.URDF` method), 14  
`MENU_IDS` (`icub_pybullet.visualizer.Visualizer.EyeWindow` attribute), 15  
module  
    `icub_pybullet.examples.push_the_ball_cartesian`, 19  
    `icub_pybullet.examples.push_the_ball_pure_joints`, 19  
    `icub_pybullet.examples.skin_test`, 20  
    `icub_pybullet.pycub`, 7  
    `icub_pybullet.utils`, 13  
    `icub_pybullet.visualizer`, 15  
`motion_done()` (`icub_pybullet.pycub.pyCub` method), 10  
`move_cartesian()` (`icub_pybullet.pycub.pyCub` method), 11  
`move_position()` (`icub_pybullet.pycub.pyCub` method), 11  
`move_velocity()` (`icub_pybullet.pycub.pyCub` method), 11

## O

`on_close()` (`icub_pybullet.visualizer.Visualizer.EyeWindow` method), 15  
`on_mouse()` (`icub_pybullet.visualizer.Visualizer.EyeWindow` method), 15

## P

`Pose` (class in `icub_pybullet.utils`), 14  
`POSITIONS` (`icub_pybullet.visualizer.Visualizer.EyeWindow` attribute), 15  
`prepare_log()` (`icub_pybullet.pycub.pyCub` method), 11  
`print_collision_info()` (`icub_pybullet.pycub.pyCub` method), 12  
`push_the_ball()` (in module `icub_pybullet.examples.push_the_ball_cartesian`), 19  
`push_the_ball()` (in module `icub_pybullet.examples.push_the_ball_pure_joints`), 19  
`pyCub` (class in `icub_pybullet.pycub`), 8

## R

`read()` (`icub_pybullet.utils.URDF` method), 14  
`read_info()` (`icub_pybullet.visualizer.Visualizer` method), 16  
`red` (`icub_pybullet.utils.CustomFormatter` attribute), 14  
`render()` (`icub_pybullet.visualizer.Visualizer` method), 16  
`reset()` (`icub_pybullet.utils.CustomFormatter` attribute), 14  
`ROOT_TAGS` (`icub_pybullet.utils.URDF` attribute), 14  
`run_vhacd()` (`icub_pybullet.pycub.pyCub` method), 12

## S

`save_image()` (`icub_pybullet.visualizer.Visualizer.EyeWindow` method), 15  
`save_image()` (`icub_pybullet.visualizer.Visualizer.MenuCallback` method), 16  
`save_images()` (`icub_pybullet.visualizer.Visualizer.EyeWindow` method), 15  
`scale_bbox()` (`icub_pybullet.pycub.pyCub` static method), 12  
`set_attribute()` (`icub_pybullet.utils.Config` method), 13  
`show_first()` (`icub_pybullet.visualizer.Visualizer` method), 17  
`show_mesh()` (`icub_pybullet.visualizer.Visualizer` method), 17  
`stop_robot()` (`icub_pybullet.pycub.pyCub` method), 12

## T

`to_string()` (`icub_pybullet.utils.Pose` method), 14  
`toggle_gravity()` (`icub_pybullet.pycub.pyCub` method), 12

## U

`update_simulation()` (`icub_pybullet.pycub.pyCub` method), 12  
`URDF` (class in `icub_pybullet.utils`), 14

## V

`Visualizer` (class in `icub_pybullet.visualizer`), 15  
`Visualizer.EyeWindow` (class in `icub_pybullet.visualizer`), 15  
`Visualizer.MenuCallback` (class in `icub_pybullet.visualizer`), 16  
`visualShapeData` (`icub_pybullet.pycub.pyCub` attribute), 12

## W

`wait_for_dialog_completion()` (`icub_pybullet.visualizer.Visualizer.MenuCallback` method), 16

`wait_motion_done()` (*icub\_pybullet.pycub.pyCub*  
*method*), [12](#)  
`write_attr()` (*icub\_pybullet.utils.URDF method*), [14](#)  
`write_urdf()` (*icub\_pybullet.utils.URDF method*), [15](#)

## Y

`yellow` (*icub\_pybullet.utils.CustomFormatter attribute*),  
[14](#)